

Inverse

乘法逆元

本文介绍模意义下乘法运算的逆元（Modular Multiplicative Inverse），并介绍如何使用扩展欧几里德算法（Extended Euclidean algorithm）求解乘法逆元。

定义

如果一个线性同余方程 $ax \equiv 1 \pmod{b}$ ，则 x 称为 a 的逆元，记作 a^{-1} 。

如何求逆元

扩展欧几里得法

▼ 实现



C++Python

```
1 void exgcd(int a, int b, int& x, int& y) {
2     if (b == 0) {
3         x = 1, y = 0;
4         return;
5     }
6     exgcd(b, a % b, y, x);
7     y -= a / b * x;
8 }
```

```
1 def exgcd(a, b):
2     if b == 0:
3         x = 1
4         y = 0
5         return x, y
6     x1, y1 = exgcd(b, a % b)
7     x = y1
8     y = x1 - (a // b) * y1
9     return x, y
```

扩展欧几里得法和求解 [线性同余方程](#) 是一个原理，在这里不展开解释。

快速幂法

证明

因为；

所以（根据 [费马小定理](#)）；

所以。

然后我们就可以用快速幂来求了。

▼ 实现



C++Python

```
1 int qpow(long long a, int b) {
2     int ans = 1;
3     a = (a % p + p) % p;
4     for (; b; b >>= 1) {
5         if (b & 1) ans = (a * ans) % p;
6         a = (a * a) % p;
7     }
8     return ans;
9 }
```

```
1 def qpow(a, b):
2     ans = 1
3     a = (a % p + p) % p
4     while b:
5         if b & 1:
6             ans = (a * ans) % p
7         a = (a * a) % p
8         b >>= 1
9     return ans
```

注意：快速幂法使用了 [费马小定理](#)，要求 p 是一个素数；而扩展欧几里得法只要求 a 与 p 互质。

线性求逆元

求出 1 到 $p-1$ 中每个数关于质数 p 的逆元。

如果对于每个数进行单次求解，以上两种方法就显得慢了，很有可能超时，所以下面来讲一下如何线性 ($O(p)$) 求逆元。

首先，很显然的；

▼ 证明

对于 a ，有 $a \cdot a^{-1} \equiv 1 \pmod{p}$ 恒成立，故在 $\pmod{p}$ 下的逆元是 a^{-1} ，而这是推算出其他情况的基础。

其次对于递归情况，我们令 a^{-1} ， a^{-1} ，有 $a \cdot a^{-1} \equiv 1 \pmod{p}$ 。再放到 $\pmod{p}$ 意义下就会得到：

当 a 为质数时，可知 a^{-1} 在模 p 意义下的乘法逆元必定存在。这时在上式 $a \cdot a^{-1} \equiv 1 \pmod{p}$ 的两边同时乘：

再带入 a^{-1} ，有 a^{-1} ，有：

我们注意到 a^{-1} ，而在迭代中我们完全可以假设我们已经知道了所有的模 p 下的逆元。

故我们就可以推出逆元，利用递归的形式，而使用迭代实现：

▼ 实现



C++Python

```

1 inv[1] = 1;
2 for (int i = 2; i <= n; ++i) {
3     inv[i] = (long long)(p - p / i) * inv[p % i] % p;
4 }

```

```

1 inv[1] = 1
2 for i in range(2, n + 1):
3     inv[i] = (p - p // i) * inv[p % i] % p

```

使用 来防止出现负数。

▼ 当 p 不为质数

[线性同余方程](#) 中指出，如果 与 不互素时不存在相应的逆元。如果 不为质数，则至少有一个 的逆元不存在。此时这个建立在递推式上的方法就不能保证结果的正确性。例如当 且 时，根据递推式，逆元需要从 $\text{inv}[p \% i]$ 即 $\text{inv}[2]$ 计算。而 2 在模 8 意义下不存在逆元， $\text{inv}[2]$ 的值是未定义的（准确来讲，递推时 $\text{inv}[2]$ 依赖于 $\text{inv}[0]$ 的初始值）。因此 3 在模 8 意义下的乘法逆元便无法正确求出。

另外，根据线性求逆元方法的式子：

递归求解，直到 返回 。

中间优化可以加入一个记忆化来避免多次递归导致的重复，这样求 中所有数的逆元的时间复杂度仍是 。

注意：如果用以上给出的式子递归进行单个数的逆元求解，目前已知的时间复杂度的上界为 ，具体请看 [知乎讨论](#)。算法竞赛中更好地求单个数的逆元的方法有扩展欧几里得法和快速幂法。

线性求任意 n 个数的逆元

上面的方法只能求 到 的逆元，如果要求任意给定 个数 () 的逆元，就需要下面的方法：

首先计算 个数的前缀积，记为 ，然后使用快速幂或扩展欧几里得法计算 的逆元，记为 。

因为 是 个数的积的逆元，所以当我们把它乘上 时，就会和 的逆元抵消，于是就得到了 到 的积逆元，记为 。

同理我们可以依次计算出所有的 ，于是 就可以用 求得。

所以我们就在 的时间内计算出了 个数的逆元。

▼ 实现



C++Python

```

1 s[0] = 1;
2 for (int i = 1; i <= n; ++i) s[i] = s[i - 1] * a[i] % p;
3 sv[n] = qpow(s[n], p - 2);
4 // 当然这里也可以用 exgcd 来求逆元, 视个人喜好而定.
5 for (int i = n; i >= 1; --i) sv[i - 1] = sv[i] * a[i] % p;
6 for (int i = 1; i <= n; ++i) inv[i] = sv[i] * s[i - 1] % p;

```

```
1 s[0] = 1
2 for i in range(1, n + 1):
3     s[i] = s[i - 1] * a[i] % p
4 sv[n] = qpow(s[n], p - 2)
5 # 当然这里也可以用 exgcd 来求逆元, 视个人喜好而定.
6 for i in range(n, 0, -1):
7     sv[i - 1] = sv[i] * a[i] % p
8 for i in range(1, n + 1):
9     inv[i] = sv[i] * s[i - 1] % p
```

逆元练习题

[乘法逆元](#)

[乘法逆元 2](#)

[「NOIP2012」同余方程](#)

[「AHOI2005」洗牌](#)

[「SDOI2016」排列计数](#)

本页面最近更新：2024/10/21 00:50:38, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[iamtwz](#), [abc1763613206](#), [buggg-hfc](#), [Chrogeek](#), [Enter-tainer](#), [Great-designer](#), [Henry-ZHR](#), [hqztrue](#), [hsfzLZH1](#), [ImpleLee](#), [lr1d](#), [JellyGoat](#), [jifbt](#), [lhxhxxxx](#), [Marcythm](#), [MegaOwler](#), [Menci](#), [MioChyan](#), [n-WN](#), [ouuan](#), [PeterlitsZo](#), [Phemon](#), [shawlleyw](#), [Siyuan](#), [skr2005](#), [sshwy](#), [stevebraveman](#), [StudyingFather](#), [threreams](#), [Tiooo111](#), [Tiphereth-A](#), [WAAutoMaton](#), [Xeonacid](#), [Zhaoyangzhen](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用